

Pràctica AWS Terraform

Josep Miquel Olivares i Muñoz

https://github.com/josemiolivares/Practica_IaC/tree/Terraform

1. Alta Disponibilitat en la infraestructura

Actualment tenim un grup d'auto-escala amb una plantilla de llançament que seria capaç de proporcionar noves instàncies repartides a través de diferents grups de seguretat alternativament automàticament. Com es pot establir en el codi que va llançar almenys dos, en lloc d'un?

En el grup d'autoescalat posar un mínim de dos instàncies i un nombre desitjable de dos i un màxim de 4, per exemple:

```
# Auto Scaling Group para las instancias web
resource "aws_autoscaling_group" "web_asg" {
```

```
    vpc_zone_identifier = aws_subnet.private[*].id
    target_group_arns = [aws_lb_target_group.web_tg.arn]
    desired_capacity = 2
```

```
    min_size = 2
    max_size = 4
```

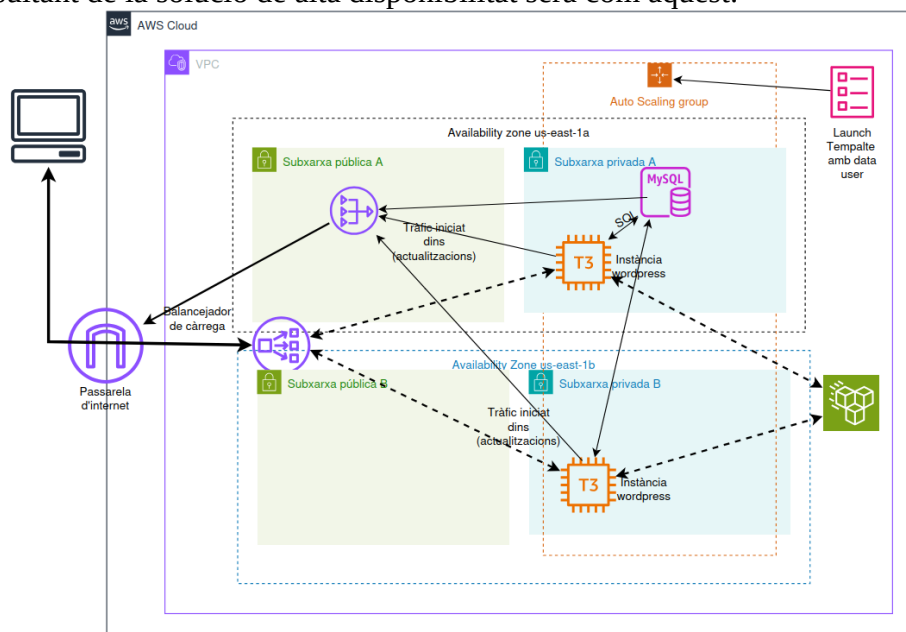
I també posar referències vectorials enlloc de posar una única amb la subxarxa actual, de les instàncies de subxarxes de forma:

```
vpc_zone_identifier = aws_subnet.private[*].id
```

També podem afegir al script de inicialització (user data) alguna cosa per veure la instància:

```
echo "<h1>Accedint a màquina WordPress</h1><br><h1>Màquina EC2: $(hostname)</h1>" >
/var/www/html/index2.html
```

El esquema resultant de la solució de alta disponibilitat serà com aquest:



D'altra banda, el servei RDS permet una configuració Multi-AZ Deployment que llança automàticament una instància de base de dades d'espera en un altre AZ, de manera transparent i sense haver de canviar l'extrem al qual es dirigeixen les sol·licituds de base de dades. Com podeu configurar el codi que realitzaria aquesta implementació?

Posant la variable `multi_az` a `true`, de forma transparent gestiona diverses instàncies de la base de dades mantenint una única connexió per a evitar duplicitats i inconsistències en la base de dades

```
resource "aws_db_instance" "wordpress_db" {  
  
  multi_az = true # Per a permetre multi-AZ transparent  
  
}
```

2. Servidor HTTPS segur amb certificat vàlid gnu gpg importat

Primer hem aconseguit del professor un subdomini amb permisos per a crear un certificat l'hem validat amb una instància DNS al subdomini. El arn del certificat i el subdomini els hem emmagatzemat con a variables.

Una primera solució pot ser habilitar el tràfic d'entrada https per el port 443 fins a les instàncies EC2 i habilitar ací els dominis amb apache2 i engegar el servei ssl per a apache2. Açò implica modificar els grups de seguretat involucrats. Una altra solució que és la que hem agafat és permetre el tràfic https fins al balancejador de càrrega i ací redirigir de forma local el tràfic del port 443 al port 80. Deixar la resta del camí fins a les instàncies EC2 igual i habilitar del certificat SSL en el balancejador de càrrega amb les polítiques adequades.

Cal deshabilitar el `http_listener` del alb de la configuració anterior i canviar-la per:

```
resource "aws_lb_listener" "https" {  
  load_balancer_arn = aws_lb.app_lb.arn  
  port = 443  
  protocol = "HTTPS"  
  ssl_policy = "ELBSecurityPolicy-TLS13-1-2-2021-06"  
  certificate_arn = var.certificate_arn  
  
  default_action {  
    type = "forward"  
    target_group_arn = aws_lb_target_group.web_tg.arn  
  }  
}
```

Que usa un arn d'un certificat autogenerat sense DNS associada, que decodifica el tràfic https a http cap a les instàncies t3 de EC2.

Si volem forçar a usar https; afegim el següent listener:

```
resource "aws_lb_listener" "http" {
  load_balancer_arn = aws_lb.app_lb.arn
  port = 80
  protocol = "HTTP"
```

```
  default_action {
    type = "redirect"
```

```
  redirect {
    protocol = "HTTPS"
    port = "443"
    status_code = "HTTP_301"
  }
}
```

El certificat el importem des de la plataforma web de AWS i declarem en seu arn com:

```
variable "certificate_arn" {
  type = string
  description = "Certificat importat de gnu pgp"
  default = "arn:aws:acm:us-east-1:143245743241:certificate/bfabf487-cb20-4219-814c-390ede6e131c"
}
```

Per a veure el balancejat podem afegir un script a la plantilla del autoscaling com:

```
<h1>Servidor EC2</h1>
<div class="info">
<p><strong>Hostname :</strong> $(hostname)</p>
<p><strong>Instance ID :</strong> $(curl -X PUT "http://169.254.169.254/latest/api/token" \
-H "X-aws-ec2-metadata-token: 21600" -s)</p>
<p><strong>Availability Zone :</strong> $(curl -H "X-aws-ec2-metadata-token: $TOKEN" \
http://169.254.169.254/latest/meta-data/placement/availability-zone -s)</p>
<p><strong>IP privada :</strong> $(curl -H "X-aws-ec2-metadata-token: $TOKEN" \
http://169.254.169.254/latest/meta-data/local-ipv4 -s)</p>
</div>
```

On es veu que s'usen eventualment els dos EC2

Servidor EC2

Hostname A: ip-10-0-101-76.ec2.internal
Instance ID B: AQAEAE5f9Q8xV38FV9guHz8AG9GSR0G8MJpBjokHb-zbxec7d9s8A==
Availability Zone C: us-east-1a
IP privada D: 10.0.101.76

Insecur tfaws-josemi-alb-1217930717.us-east-1.elb.amazonaws.com

Terraform en AWS

Mindblown: a blog about phi

I accedim al full predeterminat de wordpress, redirigint a https en qualsevol cas.